

Análise de Longevidade



Universidade Federal do Ceará - Campus de Quixadá

Lucas Ismaily
ismailybf@ufc.br

Semestre 2025.1

Compiladores

Baseado nos slides do Prof. Sandro Rigo (IC-Unicamp)

Live-variable analysis

- Linguagem intermediária

Live-variable analysis

- Linguagem intermediária
 - Gerada pelo *front-end* considerando número infinito de registradores para temporários.

Live-variable analysis

- Linguagem intermediária
 - Gerada pelo *front-end* considerando número infinito de registradores para temporários.
- Máquinas reais tem um número finito de registradores.

Live-variable analysis

- Linguagem intermediária
 - Gerada pelo *front-end* considerando número infinito de registradores para temporários.
- Máquinas reais tem um número finito de registradores.
 - 32 é um número típico para máquinas RISCs

Live-variable analysis

- Linguagem intermediária
 - Gerada pelo *front-end* considerando número infinito de registradores para temporários.
- Máquinas reais tem um número finito de registradores.
 - 32 é um número típico para máquinas RISCs
- Dois valores temporários podem ocupar o mesmo registrador se não estão “em uso” ao mesmo tempo.

Live-variable analysis

- Linguagem intermediária
 - Gerada pelo *front-end* considerando número infinito de registradores para temporários.
- Máquinas reais tem um número finito de registradores.
 - 32 é um número típico para máquinas RISCs
- Dois valores temporários podem ocupar o mesmo registrador se não estão “em uso” ao mesmo tempo.
 - Muitos temporários podem caber em poucos registradores.

Live-variable analysis

- Linguagem intermediária
 - Gerada pelo *front-end* considerando número infinito de registradores para temporários.
- Máquinas reais tem um número finito de registradores.
 - 32 é um número típico para máquinas RISCs
- Dois valores temporários podem ocupar o mesmo registrador se não estão “em uso” ao mesmo tempo.
 - Muitos temporários podem caber em poucos registradores.
 - Os que não couberem vão para a memória (*spill*).

Live-variable analysis

- Dizemos que uma variável está viva em um ponto p se ela pode vir a ser usada no futuro em um caminho a partir de p .

Live-variable analysis

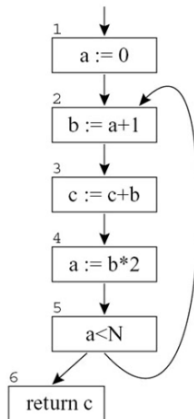
- Dizemos que uma variável está viva em um ponto p se ela pode vir a ser usada no futuro em um caminho a partir de p .
- O compilador analisa a RI para saber quais variáveis estão vivas ao mesmo tempo.

Live-variable analysis

- Dizemos que uma variável está viva em um ponto p se ela pode vir a ser usada no futuro em um caminho a partir de p .
- O compilador analisa a RI para saber quais variáveis estão vivas ao mesmo tempo.
- Esta tarefa então, é conhecida como *live-variable analysis*, ou *liveness analysis* (análise de longevidade)

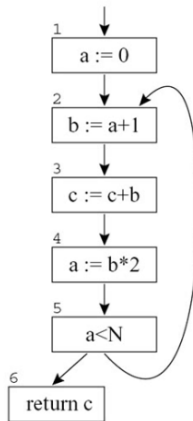
Exemplo

$a \leftarrow 0$
 $L_1 : b \leftarrow a + 1$
 $c \leftarrow c + b$
 $a \leftarrow b * 2$
if $a < N$ goto L_1
return c



Exemplo

Em quais pontos do CFG a variável b está viva?



Exemplo

Quanto registradores precisamos para comportar as variáveis do programa anterior???

Live-variable analysis

- $\text{In}[B]$: conjunto de variáveis vivas no início de B.

Live-variable analysis

- $\text{In}[B]$: conjunto de variáveis vivas no início de B.
- $\text{Out}[B]$: conjunto de variáveis vivas no final de B.

Live-variable analysis

- $\text{In}[B]$: conjunto de variáveis vivas no início de B.
- $\text{Out}[B]$: conjunto de variáveis vivas no final de B.
- def_B : conjunto de variáveis atribuídas em B antes de qualquer uso daquela variável em B.

Live-variable analysis

- $In[B]$: conjunto de variáveis vivas no início de B.
- $Out[B]$: conjunto de variáveis vivas no final de B.
- def_B : conjunto de variáveis atribuídas em B antes de qualquer uso daquela variável em B.
- use_B : conjunto de variáveis utilizadas em B antes de qualquer atribuição à variável em B.

B_2	$b = a + 1$ $c = c + b$ $a = b + 2$ if $a < N$ goto B_2	$def_{B_2} =$ $use_{B_2} =$
-------	--	------------------------------------

Computando *Liveness*

- Se a variável v está em use_B , então v é *live-in* em B e pertence a $in[B]$.

Computando *Liveness*

- Se a variável v está em use_B , então v é *live-in* em B e pertence a $in[B]$.
- Se a variável v é *live-in* no bloco B , então ela é *live-out* para todo bloco P que precede B .

Computando *Liveness*

- Se a variável v está em use_B , então v é *live-in* em B e pertence a $in[B]$.
- Se a variável v é *live-in* no bloco B , então ela é *live-out* para todo bloco P que precede B .
- Se a variável v é *live-out* no bloco B , e não está em def_B , então v é também *live-in* em B .

Equações da DFA

- Computamos $use(gen)$ e $def(kill)$ para cada B como visto anteriormente.

Equações da DFA

- Computamos $use(gen)$ e $def(kill)$ para cada B como visto anteriormente.
- Neste caso, $IN[B]$ é definido em função de $OUT[B]$

Equações da DFA

- Computamos $use(gen)$ e $def(kill)$ para cada B como visto anteriormente.
- Neste caso, $IN[B]$ é definido em função de $OUT[B]$
- Temos:

$$in[EXIT] = \emptyset$$

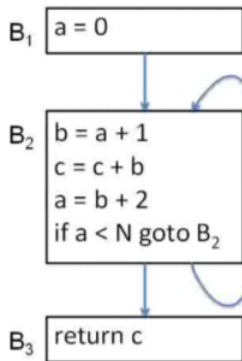
Para todo bloco B diferente de $EXIT$

$$in[B] = use_B \cup (out[B] - def_B)$$

$$out[B] = \bigcup_{S \in \text{sucessores de } B} in[S]$$

Exemplo

Exemplo



$in[B_1] =$

$def_{B_1} =$

$use_{B_1} =$

$out[B_1] =$

$in[B_2] =$

$def_{B_2} =$

$use_{B_2} =$

$out[B_2] =$

$in[B_3] =$

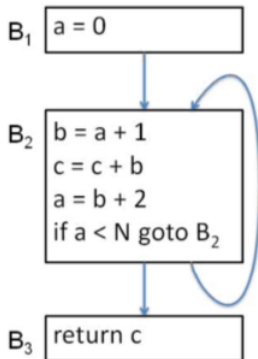
$def_{B_3} =$

$use_{B_3} =$

$out[B_3] =$

Exemplo

Exemplo



$in[B_1] = \{c\}$
 $def_{B_1} = \{a\}$
 $use_{B_1} = \{\}$
 $out[B_1] = \{a, c\}$

$in[B_2] = \{a, c\}$
 $def_{B_2} = \{b\}$
 $use_{B_2} = \{a, c\}$
 $out[B_2] = \{a, c\}$

$in[B_3] = \{c\}$
 $def_{B_3} = \{\}$
 $use_{B_3} = \{c\}$
 $out[B_3] = \{\}$

Diferenças para as anteriores

- Direção da análise

Diferenças para as anteriores

- Direção da análise
 - O conjunto $IN[B]$ é definido em função de $OUT[B]$

Diferenças para as anteriores

- Direção da análise
 - O conjunto $IN[B]$ é definido em função de $OUT[B]$
 - *Backward*: $in = f_s(out)$

Solução Iterativa

$IN[EXIT] = \{\};$ // Conjunto vazio

for (each basic block B other than $EXIT$)

$IN[B] = \{\};$

while (changes to any IN occur)

for (each basic block B other than $EXIT$) {

$$out[B] = \bigcup_{S \in \text{sucessores de } B} in[S]$$

$$in[B] = use_B \cup (out[B] - def_B)$$

}

Complexidade do Algoritmo

Qual é a complexidade do algoritmo?

```
IN[EXIT] = {}; // Conjunto vazio
for (each basic block  $B$  other than EXIT)
    IN[B] = {};
while (changes to any IN occur)
    for (each basic block  $B$  other than EXIT) {
         $out[B] = \bigcup_{S \in \text{sucessores de } B} in[S]$ 

         $in[B] = use_B \cup (out[B] - def_B)$ 
    }
```

Def-Use Chain

- Liga cada definição aos usos que alcança.

Def-Use Chain

- Liga cada definição aos usos que alcança.
- Pode ser calculada a partir das *use-def chains* (mais comum). *Use-def chains* são calculadas a partir de *reaching definitions*.

Def-Use Chain

- Liga cada definição aos usos que alcança.
- Pode ser calculada a partir das *use-def chains* (mais comum). *Use-def chains* são calculadas a partir de *reaching definitions*.
- Pode ser definida como um DFA com as mesmas equações de *live-variable analysis*

Def-Use Chain

- $OUT[B]$: usos alcançáveis a partir do final de B.

Def-Use Chain

- $OUT[B]$: usos alcançáveis a partir do final de B.
- use_B : usos $u_i(x)$ que aparecem antes de qualquer definição não ambígua à variável x .

Def-Use Chain

- $OUT[B]$: usos alcançáveis a partir do final de B .
- use_B : usos $u_i(x)$ que aparecem antes de qualquer definição não ambígua à variável x .
- def_B : todos os usos $u_i(x)$ para as variáveis x que forem definidas no bloco básico.

Def-Use Chain

- $OUT[B]$: usos alcançáveis a partir do final de B .
- use_B : usos $u_i(x)$ que aparecem antes de qualquer definição não ambígua à variável x .
- def_B : todos os usos $u_i(x)$ para as variáveis x que forem definidas no bloco básico.
- Equações e algoritmo são idênticos.

Grafo de interferência

- A informação de *liveness* é usada para otimização

Grafo de interferência

- A informação de *liveness* é usada para otimização
 - Alocação de registradores.

Grafo de interferência

- A informação de *liveness* é usada para otimização
 - Alocação de registradores.
- Interferência: ocorre quando a e b não podem ocupar o mesmo registrador

Grafo de interferência

- A informação de *liveness* é usada para otimização
 - Alocação de registradores.
- Interferência: ocorre quando a e b não podem ocupar o mesmo registrador
 - *Live ranges* com sobreposição.

Grafo de interferência

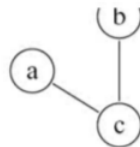
- A informação de *liveness* é usada para otimização
 - Alocação de registradores.
- Interferência: ocorre quando a e b não podem ocupar o mesmo registrador
 - *Live ranges* com sobreposição.
 - a não pode ser alocada a $r1$.

Grafo de interferência

Representação

	a	b	c
a			x
b			x
c	x	x	

(a) Matrix



(b) Graph

Grafo de interferência

- MOVE: é importante não criar falsas interferências entre a fonte e destino.

$t \leftarrow s$	<i>(copy)</i>
$\vdots$	
$x \leftarrow \dots s \dots$	<i>(use of s)</i>
$\vdots$	
$y \leftarrow \dots t \dots$	<i>(use of t)</i>

Grafo de interferência

- MOVE: é importante não criar falsas interferências entre a fonte e destino.

$t \leftarrow s$	<i>(copy)</i>
$\vdots$	
$x \leftarrow \dots s \dots$	<i>(use of s)</i>
$\vdots$	
$y \leftarrow \dots t \dots$	<i>(use of t)</i>

- s e t estariam vivas após a instrução de cópia.

Grafo de interferência

- MOVE: é importante não criar falsas interferências entre a fonte e destino.

$t \leftarrow s$	<i>(copy)</i>
$\vdots$	
$x \leftarrow \dots s \dots$	<i>(use of s)</i>
$\vdots$	
$y \leftarrow \dots t \dots$	<i>(use of t)</i>

- s e t estariam vivas após a instrução de cópia.
- Devemos aproveitar o mesmo registrador.

Grafo de Interferência

- Definição de que não seja move:

Grafo de Interferência

- Definição de que não seja move:
 - Live-out = $b_1, \dots, b_j$

Grafo de Interferência

- Definição de que não seja move:
 - Live-out = $b_1, \dots, b_j$
 - Adicione as arestas $(a, b_1), \dots, (s, b_j)$

Grafo de Interferência

- Definição de que não seja move:
 - Live-out = $b_1, \dots, b_j$
 - Adicione as arestas $(a, b_1), \dots, (s, b_j)$
- Moves $a \leftarrow c$

Grafo de Interferência

- Definição de que não seja move:
 - Live-out = $b_1, \dots, b_j$
 - Adicione as arestas $(a, b_1), \dots, (s, b_j)$
- Moves $a \leftarrow c$
 - Live-out = $b_1, \dots, b_j$

Grafo de Interferência

- Definição de que não seja move:
 - Live-out = $b_1, \dots, b_j$
 - Adicione as arestas $(a, b_1), \dots, (s, b_j)$
- Moves $a \leftarrow c$
 - Live-out = $b_1, \dots, b_j$
 - Adicione as arestas $(a, b_1), \dots, (s, b_j)$ para os b_i 's que não são o mesmo que c .

Análise de Longevidade



Universidade Federal do Ceará - Campus de Quixadá

Lucas Ismaily
ismailybf@ufc.br

Semestre 2025.1

Compiladores

Baseado nos slides do Prof. Sandro Rigo (IC-Unicamp)